

```

1 #####
2 #####
3 #####
4 ##### RUTINA QUE LEE LOS CANALES AN0, AN1, AN2 DEL CONVERTIDOR ANALÓGICO - DIGITAL #####
5 ##### Y EL CANAL AN4 QUE CORRESPONDE AL MEDIDOR DE TEMPERTURA INTERNO. #####
6 #####
7 ##### LUEGO DE REALIZAR LAS LECTURAS, SE ESTÁ HACIENDO UNA TRANSMISIÓN DE LOS #####
8 ##### UTILIZANDO LA UART0 DE LA RASPBERRY PI PICO A TRAVES DE UN MÓDULO BLUETOOTH #####
9 ##### TIPO "HC05", SINCRONIZANDO LA TRANSMISIÓN DE DATOS SE REALIZA CON LA #####
10 ##### RECEPCIÓN DEL CARACTER "U" (0x55). #####
11 #####
12 #####
13 #####
14 #####
15 ##### UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO - FACULTAD DE INGENIERÍA #####
16 ##### DEPARTAMENTO DE CONTROL Y ROBÓTICA #####
17 #####
18 #####
19 #####
20 #####
21 ##### M. EN I. JOSÉ ANTONIO DE JESÚS ARREDONDO GARZA #####
22 ##### EMAIL: jarredon@unam.mx #####
23 #####
24 ##### AÑO 2025 #####
25 #####
26 #####
27 #####
28
29
30 from machine import Pin, Timer # Importa librería de Pines.
31 from time import sleep, sleep_ms, sleep_us # Importa librerías de retardos.
32 from machine import ADC,UART # Importa librerías del A/D y la UART.
33 import _thread # Librería para poder utilizar
34 # los 2 nucleos.
35 import struct # Librería para manejo de datos
36 # de "float" a "Little ENDIAN" y de
37 # "Little ENDIAN" a "float".
38 machine.freq(27000000) # Hacer que la CPU trabaje a 270 Mhz.
39
40
41
42
43 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
44 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
45 #####
46 ##### CONFIGURACIÓN DEL CONVERTIDOR A/D PARA PODER LEER LOS CANALES AN0, #####
47 ##### AN1, AN2 Y AN3 (TEMPERATURA), LA GENERACIÓN DE UNA ONDA CUADRADA #####
48 ##### Y LA CONFIGURACIÓN DE LA UART0 DE LA RASPBERRY PI PICO/PICO2. #####
49 #####
50 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
51 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
52
53 AN0 = ADC(Pin(26)) # Define el objeto ADC (GPI26) del canal AN0.
54 AN1 = ADC(Pin(27)) # Define el objeto ADC (GPI27) del canal AN1.
55 AN2 = ADC(Pin(28)) # Define el objeto ADC (GPI28) del canal AN2.
56 SENSOR = ADC(4) # Declara el Canal que mide la Temperatura.
57 Cte_to_Volt = (3.3/65535) # Constante para conversión a Voltios Reales.
58
59
60 #####
61 ## Variable que contendrá los ##
62 ## los bytes "Little ENDIAN" ##
63 ## que se usarán para enviar ##
64 ## a través de la UART0 ##

```

```

65 #####
66 En_Bytes = bytearray(16)
67
68
69 #####
70 ## Declara UART0 a una tasa de 460800 Bauds, ##
71 ## Tx = Pin0, Rx = Pin1. La UART0 será ##
72 ## conectada al Módulo Bluetooth HC05. ##
73 #####
74 UART0 = UART(0, baudrate = 460800, tx = Pin(0), rx = Pin(1))
75
76
77 #####
78 ## Pines Accesorio Utilizados y Frecuencia de la Onda Cuadrada ##
79 #####
80 P25 = Pin(25, Pin.OUT) # Pin nativo de la tarjeta como testigo.
81 Pin3 = Pin(3, Pin.OUT) # Pin de acción paralela (que generará una
82 # onda cuadrada).
83 FREQ = 1 # Pon la frecuencia a 1 Hz.
84 Period = (1/FREQ) # Obten el periodo.
85 Half_Period = (Period/2) # Obten el semi periodo.
86
87
88
89
90
91 #####
92 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
93 #####
94 ##### RUTINA PARA GENERAR UNA ONDA CUADRADA EN PARALELO #####
95 #####
96 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
97 #####
98 def Onda_Cuadrada():
99     while True:
100         Pin3.on() # Pon en "1" al Pin GPIO3 de laa RP2.
101         sleep(Half_Period) # Retardo del semiperiodo.
102         Pin3.off() # Pon en "0" al Pin GPIO3 de la RP2.
103         sleep(Half_Period) # Retardo del semiperiodo.
104
105
106
107
108
109 #####
110 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
111 #####
112 ##### RUTINA QUE REALIZA LA LECTURA DE DATOS Y SU TRANSMISIÓN SINCRONIZADA #####
113 ##### UTILIZANDO EL PUERTO UART0 Y EL MÓDULO BLUETOOTH HC05. #####
114 #####
115 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
116 #####
117
118 _thread.start_new_thread(Onda_Cuadrada,()) # Inicia el "thread" de la onda cuadrada.
119
120 while True:
121     P25.off() # Apaga el Led testigo.
122
123     #####
124     ## ¿Hay algún byte pendiente? ##
125     ## (En este caso Rx) ##
126     #####
127     if UART0.any() > 0:
128         LLAVE = UART0.read() # Lee el caracter recibido o
129         # caracteres recibidos.

```

```

130
131 #####
132 ##                                     ##
133 ##   Investiga si se recibió un caracter "U"   ##
134 ##                                     ##
135 #####
136 if LLAVE[0] == 85:
137     P25.on()                                # Enciende el indicador de
138                                           # transmisión.
139     Read0   = AN0.read_u16()*Cte_to_Volt    # Lee el Canal AN0 del A/D.
140     Read1   = AN1.read_u16()*Cte_to_Volt    # Lee el Canal AN1 del A/D.
141     Read2   = AN2.read_u16()*Cte_to_Volt    # Lee el Canal AN2 del A/D.
142     Valor_T = SENSOR.read_u16()*Cte_to_Volt # Lee el Canal AN3 del A/D.
143     GRADOS  = 27 - (Valor_T - 0.706)/0.001721 # Obten la Temperatura de
144                                           # la CPU (centigrados).
145     print(Read0, Read1, Read2, GRADOS)
146
147 #####
148 ##   Conversión de los "floats" a formato   ##
149 ##   "Little ENDIAN" para poder transmitir ##
150 ##   la información a MATLAB utilizando    ##
151 ##   la UART0 vía Bluetooth HC05.         ##
152 #####
153 En_Bytes = bytearray(struct.pack("f",Read0))
154 En_Bytes = En_Bytes + bytearray(struct.pack("f", Read1))
155 En_Bytes = En_Bytes + bytearray(struct.pack("f", Read2))
156 En_Bytes = En_Bytes + bytearray(struct.pack("f", GRADOS))
157
158 #####
159 ##                                     ##
160 ##   TRANSMITE LA CADENA DE BYTES OBTENIDOS   ##
161 ##                                     ##
162 #####
163
164 UART0.write(En_Bytes)
165
166 P25.off()                                # Apaga el indicador de
167                                           # transmisión.

```